I. Java Core / Theory Questions & Answers

#	Question	Answer
1	Abstract class vs Interface	Abstract class: can have abstract+concrete methods, fields, constructors. Use for related classes. Interface: method signatures (default/static allowed), contract across unrelated classes.
2	Four pillars of OOP	Encapsulation: private fields+getters/setters. Abstraction: hide implementation, show essential features. Inheritance: subclass reuses fields/methods. Polymorphism: same method, different behavior (overloading/overriding).
3	== vs .equals()	== compares references (memory). .equals() compares content (may be overridden).
4	final vs finally vs finalize	final: constant/method/class. finally: block after try/catch, always executes. finalize(): GC calls before object destruction.
5	Access modifiers	private: same class. default: same package. protected: same package + subclasses. public: everywhere.
6	static vs instance	static: belongs to class, shared. instance: belongs to object.
7	String vs StringBuilder vs StringBuffer	String: immutable. StringBuilder: mutable, not thread-safe, fast. StringBuffer: mutable, thread-safe, slower.
8	Method overloading vs overriding	Overloading: same class, same name, different params. Overriding: subclass, same signature, runtime polymorphism.
9	Constructor chaining	Call one constructor from another using this(...).
10	Checked vs Unchecked exceptions	Checked: must handle. Unchecked: runtime exceptions.
11	throw vs throws	throw: throw exception object. throws: declare exception in method signature.
12	Collections overview	List: ordered, duplicates allowed. Set: unordered, no duplicates. Map: key-value pairs, unique keys.
13	ArrayList vs LinkedList	ArrayList: fast random access, slow middle insert/delete. LinkedList: slow random access, fast middle insert/delete.
14	synchronized vs volatile	synchronized: atomicity + visibility. volatile: visibility only.
15	HashMap vs TreeMap vs LinkedHashMap	HashMap: unordered, O(1). TreeMap: sorted, O(log n). LinkedHashMap: insertion order, O(1).

II. Coding / Algorithm Questions & Approaches

#	Question	Approach / Code
1	Palindrome	Two-pointer, ignore non-alphanumeric, compare lowercase chars.
2	Array duplicates	Use HashSet to track seen numbers, return true if duplicate found.
3	Max subarray sum	Kadane's algorithm: track current max and overall max.
4	Reverse string	new StringBuilder(s).reverse().toString();
5	Reverse integer	Loop with %10 and /10 to construct reversed number.
6	Merge sorted arrays	Two-pointer merge into new array.
7	Char frequency	Map<Character,Integer>, iterate and count.
8	First non-repeating char	Map to count, iterate string to find first with count 1.
9	FizzBuzz	Loop 1-100, check %3==0 && %5==0, else %3==0, else %5==0.
10	Swap numbers without temp	a=a+b; b=a-b; a=a-b; or XOR method.
11	Remove duplicates from array	Use Set or copy unique elements to new array.
12	Factorial	Recursive: n*factorial(n-1), Iterative: loop multiply.
13	Second largest	Track max and secondMax while iterating.
14	Anagrams	Sort both strings or count char occurrences.
15	Stack implementation	Use array or LinkedList: push(), pop(), peek().

III. SQL Questions & Answers

#	Question	Answer
1	Total amount per customer	SELECT customer_id, SUM(amount) AS total_spent FROM Orders GROUP BY customer_id ORDER BY total_spent DESC;
2	Highest salary per department	SELECT department_id, MAX(salary) AS highest_salary FROM Employees GROUP BY department_id;
3	Customers who never placed order	SELECT c.customer_id FROM Customers c LEFT JOIN Orders o ON c.customer_id=o.customer_id WHERE o.order_id IS NULL;
4	Number of orders per day	SELECT order_date, COUNT(*) AS orders_count FROM Orders GROUP BY order_date;
5	INNER vs LEFT vs RIGHT vs FULL JOIN	INNER: only matches. LEFT: all left + matching right. RIGHT: all right + matching left. FULL: all rows both sides, NULL if no match.